Segundo Cerebro

Cómo construí un asistente RAG local para consultar mis notas y documentos

Octubre de 2026

1. De dónde surge el proyecto

La idea del “Segundo Cerebro” apareció por una necesidad bastante concreta: tenía información repartida entre notas y documentos, y cada vez que necesitaba recuperar un dato terminaba buscando archivo por archivo. Quería poder hacer una pregunta normal, recibir una respuesta y saber exactamente de qué documento había salido.

Para resolverlo armé un asistente que trabaja sobre mi repositorio de notas en Markdown. El sistema prepara e indexa los documentos, busca los fragmentos relacionados con cada consulta y se los entrega a un modelo de lenguaje. La respuesta aparece en un chat web, acompañada de referencias que puedo revisar.

Desde el inicio me interesaba que todo quedara en mi propio equipo. Las consultas, los documentos y los modelos debían procesarse de manera local, sin enviar información a servicios externos. Esa condición terminó influyendo bastante en la arquitectura y también en los límites de rendimiento que acepté.

1.1. Por qué elegí RAG en vez de entrenar un modelo

Antes de empezar consideré dos alternativas. Una era ajustar un modelo con mis documentos mediante fine-tuning; la otra, utilizar RAG (Retrieval-Augmented Generation), que consiste en buscar la información relevante al momento de hacer cada pregunta.

El fine-tuning no me convenía para este caso. Además de exigir más recursos de cómputo, actualizar el conocimiento con documentos nuevos sería poco práctico. Tampoco me resolvía bien un requisito importante: poder comprobar de dónde salía cada respuesta.

Con RAG puedo incorporar cambios reindexando los archivos, utilizar un modelo relativamente pequeño y mostrar las fuentes consultadas. Como el repositorio cambia seguido y el servidor no tiene GPU, fue la alternativa que mejor se ajustó a lo que necesitaba.

2. Cómo está armado el sistema

Separé el funcionamiento en cinco partes. Aunque cada una cumple una tarea distinta, todas se ejecutan en el mismo servidor y se comunican dentro de mi entorno local.

- Un repositorio de notas que mantengo sincronizado entre mi notebook y el servidor.
- Un proceso de conversión que deja los documentos Word y PDF en texto plano.
- Un flujo de indexación en n8n que divide el texto, genera embeddings y guarda los vectores en Qdrant.
- Un flujo de consulta, también en n8n, que recibe la pregunta, busca el contexto y lo envía al modelo local.
- Una página web sencilla que permite conversar con el asistente, revisar las fuentes y ver el estado del sistema.

Visto de principio a fin, el recorrido es este: notebook y servidor sincronizan los archivos; los documentos que lo requieren se convierten a texto; el indexador procesa ese texto y lo guarda en Qdrant. Cuando hago una pregunta desde el chat, n8n consulta Qdrant y el modelo redacta una respuesta con citas.

2.1. Equipo y plataforma

Elemento	Qué utilizo	Por qué lo elegí
Servidor	Mini PC de oficina, CPU de cuatro	Ya era un equipo apropiado para un

Elemento	Qué utilizo	Por qué lo elegí
	núcleos, 32 GB de RAM y sin GPU	laboratorio personal: silencioso y de consumo moderado.
Sistema operativo	Ubuntu Server	Me permite administrarlo por SSH sin necesitar entorno gráfico.
Servicios	Contenedores Docker	Mantengo separados n8n, Qdrant, nginx y los demás componentes.
Acceso remoto	Red privada cifrada	Puedo entrar desde el notebook o el teléfono sin publicar puertos en Internet.

La mayor limitación es la ausencia de GPU. No me impide operar el sistema, pero sí determina qué tamaño de modelo puedo usar y cuánto debo esperar por una respuesta.

2.2. Herramientas principales

Para qué	Herramienta o solución
Automatizar procesos	n8n instalado en el servidor
Guardar y buscar vectores	Qdrant
Responder las preguntas	Modelos abiertos locales de 7B y 3B parámetros
Generar embeddings	Modelo local con vectores de 768 dimensiones
Convertir documentos	LibreOffice en modo headless y pdftotext
Sincronizar archivos	Herramienta peer-to-peer
Interfaz de consulta	HTML y JavaScript, servido por nginx
Ejecutar tareas periódicas	cron

3. Cómo preparo e indexo los documentos

Esta fue una de las partes que más ajustes me exigió. Al principio parecía simple: leer un archivo, dividirlo y guardarlo como vectores. En la práctica aparecieron varios detalles que podían dejar documentos fuera de la búsqueda sin generar errores evidentes.

3.1. Primero convierto los archivos a texto

El indexador trabaja únicamente con archivos .md y .txt. Por eso los .docx y .pdf pasan antes por una conversión automática a .txt, ejecutada por scripts cada pocos minutos. Conservo tanto el original como su versión en texto.

Tomé la decisión de sacar la conversión de n8n después de un problema concreto. Un nodo que extraía texto desde un PDF mantenía la extensión .pdf en su salida; más adelante, un script de limpieza interpretó ese archivo como si fuera el PDF original y lo eliminó. Desde entonces separé las responsabilidades: los scripts convierten y n8n solo procesa texto.

3.2. División en fragmentos

Para que la búsqueda funcione, cada documento se divide en fragmentos o chunks. Hay que encontrar un equilibrio: si son muy largos mezclan demasiada información; si son muy pequeños pierden el contexto. En la etapa de consulta recorto los fragmentos que recibe el modelo a unos 900 caracteres.

Además del texto, guardo información para rastrear cada resultado: archivo de origen, ruta, carpeta, posición del fragmento, cantidad total de fragmentos y hash del documento. Esto después permite citar la fuente y actualizarla sin confundirla con otras.

3.3. Un detalle importante: los prefijos de embeddings

El modelo de embeddings que utilizo distingue entre el contenido que se indexa y la pregunta que se quiere buscar. Por eso agrego el prefijo search_document: a los fragmentos y search_query: a las consultas.

Es un detalle fácil de pasar por alto porque el sistema sigue respondiendo aunque no se usen. El problema es que las búsquedas pierden calidad. Cuando corregí esto tuve que generar nuevamente los vectores de todos los documentos, ya que los anteriores no eran equivalentes a las consultas con el nuevo formato.

3.4. Qué guardo en Qdrant

En la colección documentos utilizo vectores de 768 dimensiones y similitud coseno. Cada punto se identifica a partir de un hash de la ruta y el número de fragmento, de manera que volver a indexar un archivo actualiza sus puntos en lugar de duplicarlos.

También definí índices para los metadatos, como ruta, carpeta y hash. Con ellos puedo filtrar o eliminar los fragmentos de un documento. El campo de texto tiene un índice de búsqueda completa, que más adelante aprovecho para combinar coincidencias exactas con la búsqueda semántica.

3.5. Cómo detecto los cambios

Cada diez minutos el indexador revisa los documentos. Para evitar procesarlo todo de nuevo, compara el hash del archivo con el valor que quedó almacenado en Qdrant. Si hubo cambios, elimina los fragmentos antiguos y genera los nuevos.

Mi primera implementación comparaba el tamaño del archivo. No funcionó bien porque n8n mostraba tamaños redondeados, como “739 kB”, y una modificación pequeña podía pasar inadvertida. Cambiar esa comparación por el hash solucionó el problema.

4. Qué ocurre cuando hago una pregunta

Desde el chat escribo la pregunta tal como la haría a otra persona. El sistema genera su embedding usando search_query:, recupera inicialmente los doce fragmentos más cercanos y combina esos resultados con una búsqueda por términos exactos. Después selecciona cinco fragmentos y los incorpora al contexto que recibirá el modelo.

El modelo redacta la respuesta indicando referencias como [1] o [2]. En el chat puedo revisar esas fuentes y abrir el documento original desde el editor de notas. También le indiqué expresamente que no complete con suposiciones lo que no aparece en los fragmentos recuperados. Si no encuentra respaldo suficiente, prefiero que lo diga.

4.1. Por qué tuve que combinar dos tipos de búsqueda

La búsqueda semántica sirve para encontrar textos relacionados por su significado, pero no siempre se comporta bien con códigos, siglas o nombres específicos. Si pregunto por un documento como PRO-005, necesito que encuentre ese identificador y no solamente algo parecido en contenido.

Por eso añadí una búsqueda por palabras exactas usando el índice de texto completo de Qdrant y después combino las dos listas de resultados. Había considerado vectores dispersos nativos, pero mi colección original estaba creada con un único vector y no podía añadirlos de esa forma sin recrearla. Preferí aprovechar el índice de texto y evitar una reindexación completa en ese momento.

4.2. Tiempos de respuesta

El costo de ejecutar el modelo sin GPU se nota sobre todo en la espera. Con el modelo de 7B y cinco fragmentos de contexto, una respuesta tarda aproximadamente 100 segundos. El modelo de 3B responde más rápido, aunque a veces con menor calidad. Dejé ambos disponibles para elegir desde el chat.

Por ahora me resulta aceptable porque hago pocas consultas al día y me importa más revisar respuestas respaldadas por documentos que obtenerlas de inmediato. Si quisiera darle un uso intensivo, tendría que evaluar una GPU o trabajar con modelos más livianos.

5. Cómo clasifico los documentos nuevos

Además de consultar archivos, quería evitar que los documentos nuevos se acumularan desordenados. Para eso creé una carpeta de pendientes que un script revisa cada cinco minutos. El sistema propone una ubicación, pero la decisión final sigue siendo mía.

Preparé una segunda colección de Qdrant llamada carpetas. En ella cada carpeta tiene un vector generado a partir de una descripción de los documentos que debería contener. Cuando llega un archivo nuevo, calculo su vector y comparo los resultados con esas descripciones.

- Se identifican las tres carpetas más parecidas.
- Si la diferencia entre las dos primeras puntuaciones es menor que 0,08, considero que la elección no está clara y le pido al modelo que revise el documento.
- La sugerencia aparece en una bandeja de aprobación dentro del chat. Puedo aceptarla o corregirla.
- Solo después de mi confirmación se mueve el archivo, junto con la versión convertida.

Al probar el clasificador encontré puntuaciones muy próximas entre sí, normalmente entre 0,67 y 0,71. Yo había establecido un umbral de 0,80 para automatizar con confianza. Podría haber bajado ese valor, pero habría ocultado el problema. Revisé las descripciones de las carpetas y descubrí que eran demasiado generales. Mejorarlas fue más útil que relajar el criterio de clasificación.

6. Seguridad y privacidad

La privacidad no quedó como un agregado al final. Fue una condición que traté de mantener en cada componente, especialmente después de detectar un problema en una versión anterior del sistema.

- Los modelos, Qdrant, n8n y la interfaz se ejecutan en el servidor local.
- Los servicios no quedan publicados directamente en la red local ni en Internet; el acceso se realiza por la red privada.
- El chat registra las aprobaciones, pero el movimiento de los archivos lo realiza el script correspondiente.
- Cualquier acción que cambie el repositorio necesita una confirmación previa.
- Realizo respaldos diarios de Qdrant y de la base de datos de n8n, y ya probé el procedimiento de restauración.
- El repositorio tiene control de versiones con git, commits automáticos cada diez minutos y el historial adicional de la herramienta de sincronización.

6.1. El problema que me hizo pasar a modelos locales

Durante los primeros meses utilicé un modelo en la nube mediante API para generar las respuestas. Al revisar el flujo descubrí que cada pregunta podía enviar hasta 25 fragmentos de mis documentos al servicio externo. El sistema funcionaba y, justamente por eso, el problema había pasado desapercibido.

Esa revisión cambió el rumbo del proyecto. Migré la etapa de respuesta a modelos locales y dejé de dar por hecho que algo era privado solo porque la interfaz estaba alojada en mi propio equipo. Aprendí que hay que revisar qué datos transmite cada componente y a qué destino, incluso cuando la aplicación parece operar normalmente.

7. Problemas que encontré y qué aprendí

Varias de las mejoras más importantes no salieron del diseño inicial, sino de errores que aparecieron cuando empecé a utilizar el sistema.

- Los prefijos de embeddings sí afectan los resultados. No usarlos no detiene las consultas, pero empeora la recuperación.
- El hash sirve mejor que el tamaño para reconocer archivos modificados. Los tamaños redondeados escondían cambios pequeños.
- En n8n, el último nodo del cuerpo de “Loop Over Items” debe volver al loop. Por no hacerlo, un flujo procesó solo el primer lote y terminó sin avisar; me costó una ejecución detenida durante seis horas dar con la causa.
- En los nodos Code de mi entorno n8n no podía utilizar fetch como esperaba. Tuve que usar this.helpers.httpRequest. Una llamada encerrada en try/catch fallaba sin hacerse evidente y la primera versión de mi búsqueda híbrida estuvo sin funcionar correctamente durante meses.
- n8n guarda datos de ejecuciones exitosas. Al no configurar la limpieza automática, el historial terminó ocupando el espacio del disco.
- Conviene convertir los PDF fuera del flujo principal y tratar por separado originales y textos extraídos.
- La estructura inicial de una colección de Qdrant condiciona lo que se puede agregar después; no haber previsto los vectores con nombre limitó la incorporación de vectores dispersos.
- Un clasificador puede entregar puntuaciones razonables y aun así no ser suficientemente confiable para actuar solo. Primero tuve que mejorar las referencias de las carpetas.
- La privacidad debe comprobarse revisando las conexiones reales hacia servicios externos.
- También hay que pensar en los cortes de energía: el equipo no volvía a encender automáticamente hasta que configuré la recuperación en BIOS.

8. En qué estado está y qué me falta

El plan original incluía cinco tareas. Ya terminé la revisión de seguridad y limpieza de servicios que habían quedado sin uso; el control de versiones; la incorporación de metadatos y la eliminación de vectores de archivos borrados; y la clasificación con aprobación junto con el RAG completamente local.

La tarea pendiente es el generador de borradores. La idea es dejar una solicitud en una carpeta para que el sistema busque la información necesaria en el repositorio y prepare un primer texto que luego pueda revisar.

También quiero observar el clasificador durante un par de semanas antes de ajustar definitivamente sus criterios, fijar las versiones de las imágenes Docker en lugar de depender de latest y probar respuestas en streaming. Esto último no necesariamente acortará los cerca de 100 segundos de procesamiento, pero permitiría comenzar a leer antes de que el modelo termine.

9. Términos que utilizo en el documento

Término	Qué significa en este proyecto
RAG	Buscar fragmentos de documentos antes de pedir al modelo que formule una respuesta.
Embedding	Representación numérica del texto que permite buscar contenido relacionado.
Chunk	Fragmento del documento que se indexa y luego puede recuperarse.
Base vectorial	Base especializada en buscar vectores similares; aquí utilizo Qdrant.
Similitud coseno	Forma de comparar vectores según el ángulo entre ellos.
Búsqueda híbrida	Combinación de coincidencias semánticas y coincidencias por términos exactos.
Payload	Información asociada a cada vector, como archivo, ruta y número de

Término	Qué significa en este proyecto
	fragmento.